

UNIT 3: FUNCTIONS & POINTERS

"The soul of C Programming lies in Functions and Pointers. Master these, and you master the machine."



Why is this the MOST IMPORTANT Unit?

Until now, you've written linear programs in `main()`. As programs grow to thousands of lines, writing everything in `main()` becomes messy, unreadable, and impossible to debug. Functions allow us to break programs into small, manageable chunks. Pointers give us direct control over the computer's memory (RAM), which is what makes C so incredibly fast and powerful.

PART 1: Functions in C

1.1 What is a Function?

A function is a self-contained block of code designed to perform a specific task. Think of it like a "worker" you hire. You give the worker some raw materials (arguments/inputs), the worker does their job (execution), and hands you back a finished product (return value).

Need / Advantages of Functions:

- **Reusability:** Write the code once, use it a thousand times without rewriting.
- **Modularity:** Breaks a large, complex problem into smaller, solvable sub-problems (Divide & Conquer).
- **Readability:** Makes the `main()` function clean and easy to understand.
- **Easy Debugging:** If a specific feature fails (e.g., the calculator's addition is wrong), you only check the `add()` function, not the whole program.

1.2 The Three Pillars of a Function

To use a function in C, you need three things:

1. *Function Declaration (Prototype): Tells the compiler about the function's name, return type, and parameters *before* it is used.*
2. *Function Definition: The actual body/logic of the function.*
3. *Function Call: Executing the function when needed.*

```
// 1. Declaration (Prototype)
int addNumbers(int a, int b);

int main() {
    int sum;
    // 3. Function Call
    sum = addNumbers(10, 20);
    printf("Sum = %d\n", sum);
    return 0;
}

// 2. Function Definition
int addNumbers(int a, int b) {
    int result = a + b;
    return result;
}
```

Output:

Sum = 30

Line-by-Line Explanation:

- `int addNumbers(int a, int b);` -> We promise the compiler: "Hey, later I will define a function called `addNumbers`. It will take two integers as input and return one integer as output."
- `sum = addNumbers(10, 20);` -> The program pauses `main()`. The values 10 and 20 (Actual Arguments) are passed to the function.
- `int addNumbers(int a, int b) { ... }` -> The values 10 and 20 are copied into `a` and `b` (Formal Arguments). We add them, store in `result`, and send it back using `return`.

1.3 Types of Functions (Based on Arguments and Return Values)

Functions are categorized into 4 types depending on whether they take inputs and whether they give an output.

Type	Description	Syntax Example
1. No Argument, No Return	Takes nothing, returns nothing. Usually just prints something.	<pre>void printGreeting() { printf("Hello!"); }</pre>
2. Argument, No Return	Takes inputs, performs an action, but doesn't hand back data.	<pre>void printSquare(int n) { printf("%d", n*n); }</pre>
3. No Argument, With Return	Takes no inputs, but generates data to return.	<pre>float getPi() { return 3.14159; }</pre>
4. Argument, With Return	The standard function. Takes inputs, processes them, returns result.	<pre>int multiply(int a, int b) { return a*b; }</pre>

PART 2: Passing Arguments (Crucial Topic!)

When you call a function and pass variables to it, how does C handle those variables? There are two ways: Call by Value and Call by Reference.

2.1 Call by Value

In Call by Value, the actual value of the variable is copied into the function's parameter. Any changes made inside the function do NOT affect the original variable.

```

void modifyValue(int x) {
    x = 100; // Changes local copy
    printf("Inside function: %d\n", x);
}

int main() {
    int a = 10;
    modifyValue(a);
    printf("Inside main: %d\n", a);
    return 0;
}

```

Output:

```

Inside function: 100
Inside main: 10

```

2.2 Call by Reference (Introduction to Pointers)

In Call by Reference, instead of passing a copy of the value, we pass the memory address of the variable using pointers. Now, the function can directly modify the original variable!

```

void modifyReference(int *x) {
    *x = 100; // Modifies original memory location!
    printf("Inside function: %d\n", *x);
}

int main() {
    int a = 10;
    modifyReference(&a); // Pass address using &
    printf("Inside main: %d\n", a);
    return 0;
}

```

Output:

```

Inside function: 100
Inside main: 100

```

The Famous "Swapping" Problem

Exam question: "Write a program to swap two numbers using Call by Value and Call by Reference. Explain the difference."

Answer: Call by Value will only swap the local copies. When the function ends, the original numbers remain unswapped. Call by Reference passes addresses, allowing the function to reach into `main()`'s memory and swap the actual variables.

PART 3: Recursion

Recursion is a technique where a function calls itself to solve smaller instances of the same problem. Think of it like looking into two facing mirrors - infinite reflections!

Every recursive function **MUST** have two parts:

1. Base Case: The condition that **STOPS** the recursion (prevents infinite loop/stack overflow).
2. Recursive Case: The part where the function calls itself with a modified, smaller parameter.

3.1 Factorial Using Recursion

Factorial of N ($N!$) = $N \times (N-1)!$. For example, $5! = 5 \times 4!$. The base case is $0! = 1$ or $1! = 1$.

```
int factorial(int n) {
    // Base Case
    if (n == 0 || n == 1) {
        return 1;
    }
    // Recursive Case
    return n * factorial(n - 1);
}

int main() {
    printf("Factorial of 4 is: %d", factorial(4));
    return 0;
}
```

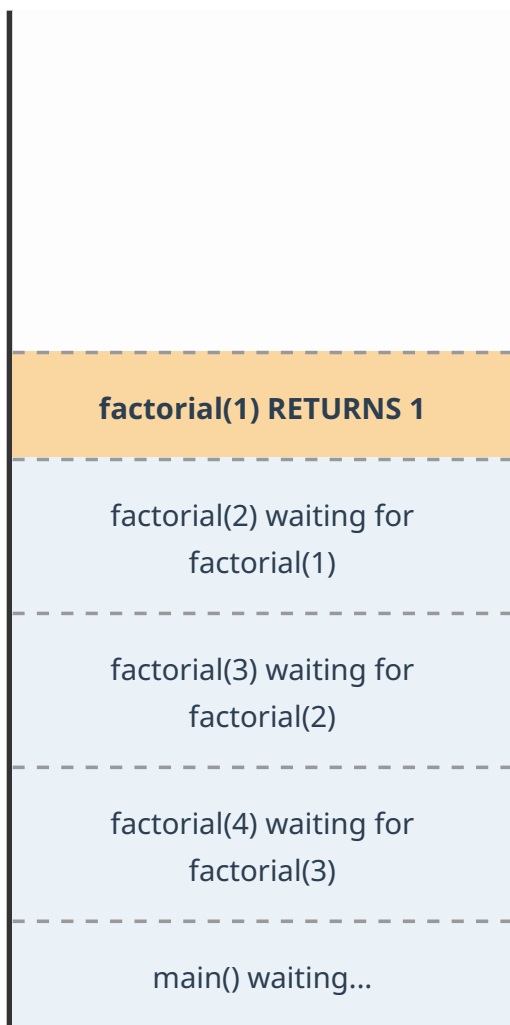
Output:

Factorial of 4 is: 24

3.2 Internal Working: Stack Memory Visualization

When a function calls itself, the computer uses a data structure called the *Call Stack* to keep track of the suspended functions. Let's trace `factorial(4)`.

Call Stack Growth for factorial(4)



Once base case (1) is hit, it returns to `fact(2)`.

`fact(2)` calculates $2 * 1 = 2$, returns to `fact(3)`.

`fact(3)` calculates $3 * 2 = 6$, returns to `fact(4)`.

`fact(4)` calculates $4 * 6 = 24$, returns to `main()`.

3.3 Fibonacci Series Using Recursion

Fibonacci: 0, 1, 1, 2, 3, 5, 8... where $F(n) = F(n-1) + F(n-2)$.

```
int fibonacci(int n) {
    if (n == 0) return 0; // Base case 1
    if (n == 1) return 1; // Base case 2
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

PART 4: Storage Classes

Storage classes decide four things about a variable: Scope (where can it be used?), Lifetime (how long does it stay in memory?), Storage (where is it stored in RAM?), and Initial Value (what is inside it if uninitialized?).

Storage Class	Keyword	Storage Location	Default Value	Scope	Lifetime
Automatic	auto	Stack Memory	Garbage Value	Local (Within block)	Till end of block
Static	static	Data Segment	Zero (0)	Local	Till program ends
Register	register	CPU Register	Garbage Value	Local	Till end of block
External	extern	Data Segment	Zero (0)	Global (All files)	Till program ends

The Magic of 'static'

A static variable acts like a global variable but is hidden inside a function. It initializes only ONCE. If a function increments a static variable and ends, the variable retains its value for the next time the function is called!

PART 5: POINTERS (The Heart of C)

5.1 Pointer Basics

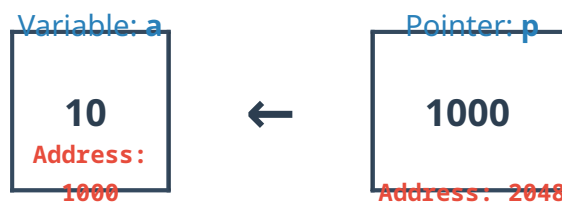
A pointer is a variable that stores the memory address of another variable. Instead of holding a value (like 5 or 'A'), it holds a location (like RAM address 2048).

- **&** (Address-of Operator): Returns the memory address of a variable.
- ***** (Value-at Operator / Dereference): Returns the value stored at a specific memory address.

```
int a = 10;
int *p = &a; // p stores the address of a

printf("Value of a: %d\n", a); // Output: 10
printf("Address of a: %p\n", &a); // Output: e.g., 0x1000
printf("Value of p: %p\n", p); // Output: 0x1000 (Same as &a)
printf("Value via pointer: %d\n", *p); // Output: 10 (Dereferencing)
```

5.2 Memory Diagram Visualization



p points to a because p contains 1000.

5.3 Special Types of Pointers

- **NULL Pointer:** A pointer that points to nothing. Initializing to `NULL` prevents accidental corruption of memory. Syntax: `int *ptr = NULL;`
- **Wild Pointer:** An uninitialized pointer. It holds a garbage address and is highly dangerous because writing to it can crash the program. Syntax: `int *ptr;`
- **Dangling Pointer:** A pointer that points to a memory location that has been deleted or freed.

5.4 Pointer Arithmetic

You can perform addition and subtraction on pointers. However, adding 1 to a pointer doesn't simply add 1 to the address number. It adds $1 * \text{sizeof}(\text{datatype})$.

```
int a = 10;
int *p = &a; // Let's assume address is 1000
p = p + 1; // Address becomes 1004 (because int is 4 bytes!)
```

5.5 Arrays and Pointers Relationship

In C, the name of an array is actually a constant pointer to its first element!

```
arr == &arr[0]
```

Because of pointer arithmetic, you can access array elements like this:

`arr[i]` is exactly the same as `*(arr + i)`

```
int arr[] = {10, 20, 30};
int *p = arr; // Points to 10

printf("%d\n", *p); // Output: 10
printf("%d\n", *(p + 1)); // Output: 20
```

5.6 Pointer to Pointer (Double Pointer)

A pointer that stores the address of another pointer. Used heavily in dynamic 2D arrays and complex data structures.

```
int a = 5;
int *p = &a; // Pointer to int
int **pp = &p; // Pointer to pointer to int

printf("%d", **pp); // Output: 5
```

PART 6: Dynamic Memory Allocation (DMA)

6.1 Static vs Dynamic Memory Allocation

Static Memory Allocation	Dynamic Memory Allocation
Memory is allocated at Compile Time.	Memory is allocated at Run Time.
Size cannot be changed (e.g., <code>int arr[100];</code>).	Size can be changed at runtime as per user need.
Stored in Stack Memory.	Stored in Heap Memory.
Memory automatically destroyed when scope ends.	Programmer must manually free memory using <code>free()</code> .

6.2 The Core DMA Functions (include <stdlib.h>)

1. `malloc()` - Memory Allocation

Allocates a single large block of contiguous memory. It contains garbage values by default.

Syntax: `ptr = (cast-type*) malloc(byte-size);`

2. `calloc()` - Contiguous Allocation

Allocates multiple blocks of memory of the same size. Crucially, it initializes all bytes to ZERO.

Syntax: `ptr = (cast-type*) calloc(n, element-size);`

Exam Table: malloc() vs calloc()

malloc()	calloc()
Takes 1 argument (total bytes).	Takes 2 arguments (number of blocks, size of each).
Default value: Garbage.	Default value: Zero.
Faster execution.	Slower execution (due to zeroing).
<code>ptr = (int*)malloc(5 * sizeof(int));</code>	<code>ptr = (int*)calloc(5, sizeof(int));</code>

3. realloc() - Re-allocation

Used to increase or decrease the size of an already dynamically allocated memory block without losing old data.

Syntax: `ptr = realloc(ptr, new-size);`

4. free() - De-allocation

Frees the dynamically allocated memory back to the heap. If you don't free memory, it causes a Memory Leak!

Syntax: `free(ptr); ptr = NULL;`

6.3 Program: Array sum using malloc()

```

#include <stdio.h>
#include <stdlib.h>

int main() {
    int n, *ptr, sum = 0;
    printf("Enter number of elements: ");
    scanf("%d", &n);

    // Dynamically allocate memory for n integers
    ptr = (int*) malloc(n * sizeof(int));

    if (ptr == NULL) {
        printf("Memory allocation failed!");
        exit(1);
    }

    // Take input and sum
    for(int i=0; i<n; ++i) {
        scanf("%d", ptr + i);
        sum += *(ptr + i);
    }

    printf("Sum = %d", sum);
    free(ptr); // Prevent memory leak
    return 0;
}

```

PART 7: Function Pointers

Just like arrays and variables have memory addresses, executable code (Functions) also sits in memory! A function pointer stores the address of a function, allowing you to pass functions as arguments or call them dynamically.

Syntax: `return_type (*pointer_name)(arg_types);`

```

void sayHello() {
    printf("Hello from function pointer!\n");
}

int main() {
    // Declare function pointer
    void (*func_ptr)() = sayHello;

    // Call function using pointer
    (*func_ptr)(); // or simply func_ptr();
    return 0;
}

```

PART 8: EXAM KIT (Quick Revision)

Important Theory Questions

1. Differentiate between Call by Value and Call by Reference with a suitable example (e.g., swapping two numbers).
2. What is recursion? Explain the working of finding the factorial of a number using memory/stack diagrams.
3. Explain storage classes in C. Discuss scope, lifetime, and default values.
4. What are Pointers? Discuss Pointer Arithmetic. Explain the relationship between 1D arrays and pointers.
5. Differentiate between `malloc()` and `calloc()`.
6. What is a Dangling pointer, Wild pointer, and Null pointer? Give examples.

Viva Questions

- Q: Why does an array index start at 0?

A: Because index is an offset. `arr[0]` means `*(arr + 0)`, which is the base address itself.

- Q: What happens if you forget to write the base condition in a recursive function?

A: Infinite recursion occurs, leading to a Stack Overflow crash.

- Q: Can we perform division or multiplication on pointers?

A: No, only addition and subtraction are allowed. Multiplying two memory addresses makes no physical sense.

- Q: What is a memory leak?

A: It happens when we dynamically allocate memory using `malloc` but forget to use `free()` before the program ends. The memory remains occupied and unusable.

Quick MCQs

1. Which operator is used to extract the value stored at the address pointed by a pointer?

- a) &
- b) *
- c) ->
- d) %

Answer: b) *

2. What is the default return type of a C function if not specified?

- a) void
- b) float
- c) int
- d) None

Answer: c) int

3. Which memory area is used for storing dynamically allocated variables?

- a) Stack
- b) Code
- c) Data
- d) Heap

Answer: d) Heap

 End of Unit 3 Notes 

Read through these concepts twice, trace the recursive trees on paper, and type out every pointer program on your compiler. Pointers are learned through fingers, not just eyes!